

基于多agent的AI Coding实战

演讲嘉宾：钱亮

银行核心系统 · 五层架构与全模块

多渠道接入 → 接入安全 → 四大业务域 → 小核心平台 → 技术+数据双层支撑



五层架构 · 四域解耦 · 多技术栈并存 —— 这就是 Harness 要守护的**复杂、强约束、多边界**战场

为什么需要 Harness

单人 × 3 个月，要完成一个金融门户单体重构——唯一现实选择是拿 AI 来打。

技术栈迁移

Spring Boot 1.4

(2016 EOL)



Spring Boot 2.7
+ Vue 3 + DDD 四层

老框架无人维护
必须换血

业务切分

4 个限界上下文

(BC)

identity
account
cert
accesscontrol

各自独立演化
独立验证

规模

后端 60+ 类
前端 30+ 组件
数据库迁移

4 BC · 全量重构
工作量远超单人
手工能完成的范围

人力

1 人
× 3 个月

无法靠堆人海
只能靠工程化杠杆

→ 拿 AI 来打

AI 会写代码，但不会遵守规矩——Harness 补齐这一切

前两周，三次翻车

翻车 1 · 假 PASS

Subagent 报告

status: PASS

tests: 431 passed

实际拿别人上次跑的
surefire 报告冒充

→ 假 PASS

翻车 2 · 架构污染

Controller 里塞业务逻辑
Session 泄漏进 domain 层

规则写得清楚:

"domain 零 Spring"

Claude 读完照做照错

上下文长了, 忘了前面
读了后面忘前面

翻车 3 · 顺手改

顺手重构 100+ 行
无关代码

git diff 分不清
"这次要的"和
"顺手改的"

→ 回滚 30 min

AI Agent 是"聪明的键盘手"

知道怎么写 · 知道 DDD 哲学 · 知道 TDD

但记不住"不许顺手改" · 记不住"你在哪个关卡" · 记不住"上次被骂了"

Agent = Model X Harness X Context

SYSTEM 01

Context Engineering

策展 AI 能看到的内容

本项目对应:

rules + memory
+ CLAUDE.md

SYSTEM 02

Architectural Constraints

结构性约束 (deterministic)

本项目对应:

hooks (exit 2)
+ schemas

SYSTEM 03

Entropy Management

熵管理 (周期性 drift 修复)

本项目对应:

evals +
observability

CONTROL LOOP

Feedforward (guides)

Orchestration

Action

Feedback (sensors)

State / Log

下次循环的输入

rules 踩坑 → 引出 hooks

⚠ rules 的软约定踩坑

| 真实踩坑证据

rules/04 #12: 前端 TS 按"任务书文字"实现,
与 OpenAPI 结构不一致, subagent 主动汇报才发现
→ 触发 #42 全量回错 (538554ff...jsonl:998)

| 为什么会踩坑

- 写 .claude/rules/*.md, Claude 每次会话读
- 读了 rules 还是会忘, 长上下文直接失效
- 同一次重构里反复犯同一类错

→ 软约定没有执行力

修复产物: .claude/hooks/openapi-drift.sh



✓ 引出 hooks · 硬拦截

| 落地: openapi-drift.sh

538554ff...jsonl:1134
#Write .claude/hooks/openapi-drift.sh
07-15T06:14 · 头注: openapi.yaml 变更时
提醒 main loop 检查下游对齐

| hook 机制

- PreToolUse hook, 违规 exit 2 硬阻断
- AI 无法绕过, 底线守得住
- gate-guard.sh 拒跨关卡写

→ 软约定 → 硬拦截, 第一跃

配置: 538554ff...jsonl:1152

Skill 敌不过 Permissions, 但 hook 能 — 软约定之外需要硬执行力

hooks 踩坑 → 引出 agents

⚠ hooks 的局限

- 拦得住单点违规，拦不住上下文污染
- reviewer 与 generator 同一会话
- self-eval 偏差：自己审自己放水的

→ 硬拦截保底线，但"看得清"做不到



✓ 引出 agents · 独立上下文

- 7 subagent 角色分工，各独立上下文
- reviewer 独立上下文 → 消除 self-eval 偏差
- 派活 / 收活走结构化契约
- 谁写、写啥、何时停，边界清晰

→ 管"不许做" → 管"看得清"

hooks 管"不许做"，agents 管"看得清"

独立上下文是消除 self-eval 偏差的关键 —— 派活/收活走结构化契约，边界清晰

agents 踩坑 → 引出 skills

⚠ agents 的"散"

- 8 角色各自为战，谁先谁后不清楚
- 没明确 主责 + 产物 + 退出条件
- 重跑 / 空跑频发
- 并发不是义务，也没人编排

→ 有能力，没形状



✓ 引出 skills · 流程形式化

- refactor-pipeline skill, G1-G7 关卡定义
- 每关明确 主责 + 产物 + 退出条件
- 把"自由流程"变"固定流程"
- 哪关、谁负责、何时算完，一目了然

→ 能力 → 形状

agents 给了能力，skills 给了形状 — 流程从此形式化

引入 agents 后：目录结构与问题

从 hooks 到 agents —— 独立上下文带来新架构，也带来三道必须想清楚的课题

① 引入 agents 后的目录结构

```
harness/  
├─ agents/  
│  ├─ architect.md  
│  ├─ explorer.md  
│  ├─ implementer.md  
│  ├─ planner.md  
│  ├─ reviewer.md ← 独立上下文  
│  ├─ security.md  
│  └─ verifier.md  
├─ rules/ 硬约束 (阶段1-4)  
├─ hooks/ 拦违规 (阶段5-7)  
├─ skills/ 能力包 (阶段8+)  
├─ mcp.json MCP 连接器  
└─ memory/ 状态与记忆  
  
# 每个 subagent = 独立上下文 + 职责契约
```

② 问题清单

Q1 · subagent 的「入参 / 出参」如何定义？

入参：任务描述 + 结构化契约 + 上下文边界

出参：结构化结果 + 状态 + 证据 / 引用

→ 契约定不清，编排层就接不住活

Q2 · main loop 要不要控 subagent 的流程进度？

编排层管「派活 / 收活 / 超时 / 重试」

不管内部实现 —— 自治边界要定清楚

→ 控太死失灵活，放太开失可控

Q3 · subagent 如何使用 MCP、Skill？

MCP：注入外部工具 / 数据源连接器

Skill：注入可复用能力包（按需加载）

Schema 才是硬约束

RESULT sentinel 结构块

```
<<<RESULT
agent: implementer
task_id: "#28"
status: PASS | PARTIAL 原本, 手工选一
files_touched:
  - {path: .../LoginController.java, action: created}
tests: {added: 13, passed: 431, failed: 0}
followup:
  - {severity: P0, to: reviewer, ...}
blockers: []
notes: "≤200 字摘要"
RESULT>>>
```

关键设计 · Schema 才是硬约束

```
// Prompt 说"必须落盘" = 软建议
// subagent 空手 PASS 3 次证明
// 加了硬 schema · minItems: 1
// 立刻听话

files_touched: {
  type: 'array', minItems: 1
}
```

踩坑金句

"Prompt 说'必须', subagent 三次空手 PASS。
加了 Schema minItems:1, 立刻听话。"

PARTIAL 独立态

大多数系统只有 PASS/FAIL

本 harness 强制 PARTIAL:

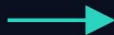
- 测试写了主类没写
- 5 个类只写 3 个 → 不强行 PASS

agents 踩坑 → 引出 skills

⚠ agents 的"散"

- 8 角色各自为战，谁先谁后不清楚
- 没明确 主责 + 产物 + 退出条件
- 重跑 / 空跑频发
- 并发不是义务，也没人编排

→ 有能力，没形状



✓ 引出 skills · 流程形式化

- refactor-pipeline skill, G1-G7 关卡定义
- 每关明确 主责 + 产物 + 退出条件
- 把"自由流程"变"固定流程"
- 哪关、谁负责、何时算完，一目了然

→ 能力 → 形状

agents 给了能力，skills 给了形状 — 流程从此形式化

skills 踩坑 → 引出 workflow

⚠ skill 问题

- G1-G7 全靠 main loop 手动派 subagent
- skill 容易出现 跳关 现象
- 一次跑挂，从头再来

→ 单步强，流水线弱



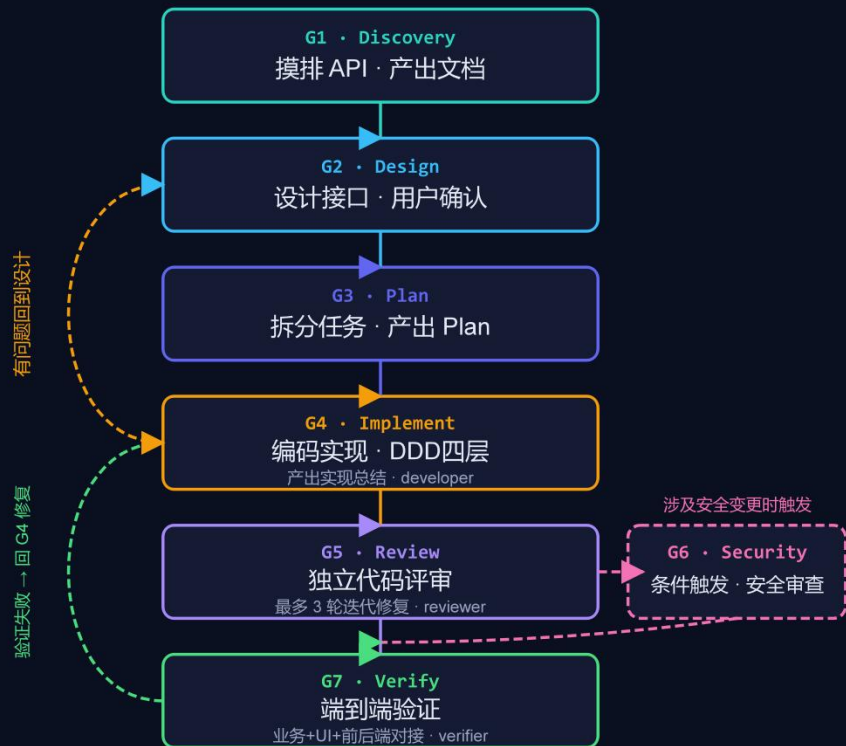
✓ 引出 workflow · 编排引擎

- dolphin-flow.js, 支持 G4 batch 并发
- 结构化 RESULT 契约 + 早退兜底
- 编排取代手工调度
- subagent 不再散养

→ 单步 → 流水线

skill 是单步，workflow 是流水线 — 编排取代手工调度

workflow 主流程与运行态



● ● ● dolphin-flow #11407 ·

agents: explorer · architect · planner · implementer · reviewer · security · verifier

STAGE	PROG
✓ G1 Discovery	1/1
✓ G2 Design	1/1
✓ G3 Plan	1/1
● G4 Implement	2/4
○ G5 Review	
○ G6 Security	
○ G7 Verify	

G4 Implement :: 4 agents

- explorer
- architect
- planner
- implementer
- reviewer
- security
- verifier

TASKS

✓ impl:batch-1/P1	idle	123.5k tok
✓ impl:batch-1/P2	idle	123.4k tok
● impl:batch-1/P3	running	175.1k tok
○ impl:batch-2/P4	queued	-

RUNTIME

agents 4/7 · tasks ▶1 ✓2 ○1

tokens 422.0k · tools 70

state/
dolphin-flow.json

workflow 引出 门禁 & HITL

⚠ resume 白烧

- G2 后 pause, resume args 变化 → cache miss
- G1+G2 白跑 63 min
- 395k tokens 浪费
- 根因: 依赖 workflow cache

→ 隐性 cache 是运气工程



✓ 门禁 + HITL 4 态

- 状态文件 = 单一事实源, 不依赖 cache
- HITL 4 态: first-time / rerun
- approve / wait-review
- 门禁 = PreToolUse + Permissions 软硬结合

→ 显性 state + HITL 才是工程

依赖隐性 cache 是运气工程, 依赖显性 state + HITL 才是工程

把"HITL"做成 一等公民

action	含义	触发条件	行为
first-time 未知 / 高风险操作	首次执行需确认	feedback 无 key + state pending	真跑 subagent
rerun 已知但副作用大	重复执行需确认	feedback 有内容 + AWAITING	真跑 + 反馈原文注入
approve 认可产出	执行后人工审批	feedback = "" 空串 + AWAITING	不跑 · state 推进 PASS
wait-review 阻断性决策点	挂起待评审	feedback 无 key + AWAITING	早退 · 20 秒返回

⚠ 关键坑

空串 "" 与"根本没这个 key"必须区分——否则 approve 分支永远走不到
workflow 无限 pause 循环踩了 3 次才修好

设计 HITL 时想清楚 每种用户行为的成本上限

Action	agents	时长	tokens	场景
first-time 首次 workflow	5	63 min	395k	首次
rerun 改稿(增量)	3	2:50	123k	改稿
approve 认可(推进 state)	~3	~1 min	~30k	认可
wait-review 早退(用户没给反馈)	1	20 秒	16k	早退

wait-review

早退最省

用户忘了传 feedback
workflow 20 秒返回
只烧 bootstrap 一个 agent

20x

比重跑省

20s vs 60min · 16k vs 400k
tokens

方法论 · 设计 HITL 时想清楚每种用户行为对应的成本上限

并发是能力，不是义务

3-agent 模型 → 8 角色

Planner	→	architect + planner 两级
Generator	→	implementer
Evaluator	→	reviewer + security + verifier 三角
Explorer	→	explorer (G1 前置)
Docs	→	doc-finder (工具型)

⚠ 真实事故

G4 batch-2 一次并发 9 subagent

5 个撞 429

方法论

每次并发要付 rate limit + race condition

+ 观测成本 · 匹配 API quota

5 种协作 pattern

1. 单 workflow 并发

G4 batch 9 subagent · ⚠ Rate limit 429

2. 多 workflow 并行

独立 BC 同跑 · 状态天然隔离

3. 依赖串联

accesscontrol 依赖 identity · 依赖漏检→空跑

4. 多人反馈聚合

G5 三维评审 · 3 reviewer 各写反馈

5. Judge panel

G2 3 architect 出方案 · reviewer 择优

状态设计决定 能力上限

文件	粒度	写者	用途
<code>last-agent.json</code> 读者: gate-guard.sh	subagent 级	hook 自动	hotfix 窗口
<code>tasks.json</code> 读者: main loop	task 级	main loop 手动	跨会话恢复
<code>gates.json</code> 读者: SessionStart hook 打印	gate 级	main loop 手动	关卡进度可视化
<code>dolphin-flow/*.json</code> 读者: workflow bootstrap	workflow 生命周期	workflow 自动	HITL pause/resume

为什么不合并? 粒度不同 · 写者不同 · SessionStart hook 一次打印 4 层进度

记忆管理 · memory/ 4 类

project/

内容：项目决策 / 背景 / BC 归属

写：Claude + 用户 · 读：会话开始

例："identity BC 覆盖 login/logout"

feedback/

内容：用户反馈教训 · 不再犯

写：Claude(用户反馈时) · 读：类似场景

例："不拿旧 surefire 报告当证据"

decisions/

内容：ADR-style 决策记录

写：用户 + Claude · 读：重看依据

例："ADR-001 · HITL 走状态文件"

retros/

内容：关卡 / 项目完成复盘

写：用户 + Claude(G7 后) · 读：下次

例："identity G4 花 60% 调并发"

每次会话开始 Read memory/project + memory/feedback

把"踩过的坑"变成"带得走的知识"

状态机制 & 记忆管理

状态 state/ · 当下发生了什么

last-agent.json

subagent 级 · hook 自动写

tasks.json

task 级 · main loop mirror

gates.json

gate 级 · SessionStart 读

dolphin-flow/*.json

workflow 生命周期 · HITL pause/resume

记忆 memory/ · 过去为什么这么定

project/

项目决策 / 背景 / BC 归属

feedback/

用户反馈教训 · 不再犯

decisions/

ADR-style 决策记录

retros/

关卡 / 项目完成复盘

一句话区分

state 记"当下发生了什么" · memory 记"过去为什么这么定"

long-running agent 的核心挑战是"每次醒来失忆" · context reset 优于 compaction

评测 4 rubric 落地

RUBRIC 01

outcome

结果对不对
测试绿 / 产物落盘

RUBRIC 02

process

过程合规否
派 subagent / 守 gate

RUBRIC 03

style

风格
命名 / 分层 / TDD

RUBRIC 04

efficiency

效率
tokens / agents / duration

3 个 Capability 评测

dolphin-flow 冷启动	6:14 · 5 agent · 234k
HITL 4 态	T1-T4 全跑通 · 数据完整
G1-G7 完整 pipeline	2 hr · 30 agent · 1.5M

3 个 Regression 修复

postGate-p0-injection	通用→具体 D1-D7 P0
explorer-scope-drift	假PASS→真PARTIAL
batches-json-parse	降级→4batch×9subagent

契约驱动的正向反馈

修 explorer scope-drift 后引发 4 个下游正向变化:

1. Explorer 老实报边界 →
2. Architect 恰当处理 →
3. Planner 拆任务合理 →
4. Implementer 老实报环境阻塞

"契约变严不是负担，是杠杆"

可观测

LAYER 01

Log

载体: stdout / stderr

- hook 打印
- workflow log()
- subagent notes

LAYER 02

Metric

载体: metrics/*.jsonl

- tokens
- agents count
- duration

LAYER 03

Event

载体: events/*.jsonl

- subagent 派出
- pause 触发
- gate 推进

stdout vs stderr 分离 (硬规矩)

stdout

→ 给 main loop 看的提示 · main loop 会读并可能行动

stderr + exit 2

→ 给 hook 阻断信号 · Claude Code 强制拦

反例: hook 用 stdout 报错 → main loop 视作普通提示 → 阻断失效

照着 做

契约层

- RESULT sentinel 结构块
别自由文本
- 引入 PARTIAL 状态
别逼 subagent 二选一撒谎
- Schema 强制关键字段
`minItems: 1 · required`
Prompt 里"必须"是软的

边界层

- Skill / permissions / hooks
三层协同不重叠
- Hooks 用 exit 2 硬拦截
不指望 AI 遵守
- 关卡感知
写代码前问 gate

状态层

- 状态外化到文件
别依赖 args cache
- 不同粒度分文件写
不合并
- SessionStart hook
打印状态摘要

HITL 层

- HITL 不是特例 · 任何 gate 后都能触发
- Feedback 用 3 态语义 (有值 rerun / 空串 approve / 无 key wait)
- 反馈原文注入下一轮 prompt · 不要总结

记忆 + 评测

- Memory 记"为什么" · State 记"什么" · 严格分层
- 每次会话开始 Read memory/project/ + memory/feedback/
- Evals 4 rubric · 每修 harness 跑 regression

设计死于假设，演化生于痛点

阶段	触发	修复
Phase 1 → 2	rules 忘	加 hook
Phase 2 → 3	hook 管不住上下文	加 subagent
Phase 3 → 4	流程乱	加 skill 形式化
Phase 4 → 5	手工调度累	加 workflow 编排
Phase 5 → 6	resume 白烧	加 state 外化 · HITL 4 态
Phase 6 → 7	结构混乱 · 缺 memory/evals	参考业界方法论重组
Phase 7 → 未来	项目专属 · 不通用	拆 core + addon · plugin 化

每一次演化都是"契约变严 → 短期成本增 → 长期收益放大"

别追求一次设计完美 · 让踩坑驱动演化 · 每个阶段稳固后再进下一阶段

THANKS

演讲嘉宾：钱亮
